第四次作业：傅里叶投影与傅里叶插值

PB23000270 李佩优

2026 年 4 月 7 日

1 引言

本作业旨在研究周期函数的傅里叶逼近方法，包括傅里叶投影（Fourier projection）和傅里叶插值（Fourier interpolation）。给定一个以 2π 为周期的函数 $f(x)$，其傅里叶级数为

$$f(x) = \sum_{k=-\infty}^{\infty} \hat{f}_k e^{ikx}, \quad \hat{f}_k = \frac{1}{2\pi}\int_0^{2\pi} f(x)e^{-ikx}dx.$$

截断至 $|k| \le K$（其中 $N = 2K$ 为总模态数）得到傅里叶投影 $P_N f$：

$$P_N f(x) = \sum_{k=-K}^{K} \hat{f}_k e^{ikx}.$$

另一方面，傅里叶插值是在 $N+1$ 个等距节点 $x_j = \frac{2\pi j}{N+1}$ 上构造一个三角多项式 $I_N f$，使得 $I_N f(x_j) = f(x_j)$，且其模态数同样为 $N+1$（$k = -K, \ldots, K$）。本文实现并比较了两种方法在两个典型周期函数上的逼近误差。

2 方法

2.1 测试函数

选取两个具有代表性的周期函数：

- $u(x) = \dfrac{1}{5 - 4\cos x}$。该函数解析且无限光滑，其傅里叶系数呈指数衰减。
- $v(x) = \sin(x/2)$。该函数在 $[0, 2\pi]$ 上连续，但其周期延拓后在端点 $x = 0, 2\pi$ 处导数不连续（左导数 $-1/2$，右导数 $1/2$）。其傅里叶系数解析式为 $\hat{v}_k = \frac{(-1)^k}{\pi(1-4k^2)}$，渐近行为 $|\hat{v}_k| \sim C/k^2$，因此属于代数衰减。

2.2 离散计算

傅里叶系数采用矩形法数值积分近似：

$$\hat{f}_k \approx \frac{1}{M}\sum_{j=1}^{M} f(x_j)e^{-ikx_j},$$

其中 $\{x_j\}$ 为 $[0,2\pi)$ 上的 M 个等距采样点，$M = 2^{\lceil \log_2(10\max(N,64))\rceil}$ 以保证积分精度。插值系数直接由 $N+1$ 个等距节点上的离散傅里叶变换给出：

$$d_k = \frac{1}{N+1}\sum_{j=0}^{N} f(x_j)e^{-ikx_j}, \quad x_j = \frac{2\pi j}{N+1}.$$

然后在精细网格 x_{fine} 上重构 $P_N f$ 和 $I_N f$，并计算逐点误差。收敛阶定义为

$$\text{Order} = \log_2\left(\frac{\text{Err}(N)}{\text{Err}(2N)}\right),$$

其中 $\text{Err}(N)$ 为 L^∞ 误差（最大绝对误差）。

2.3 实验设置

取 $N = 4, 8, 16, 32, 64$（对应 $K = N/2$）。对每个 N，计算投影误差 $\varepsilon_P(x) = |f(x) - P_N f(x)|$ 和插值误差 $\varepsilon_I(x) = |f(x) - I_N f(x)|$，并在半对数坐标下绘制误差曲线。同时绘制 $P_N f$ 和 $I_N f$ 本身，观察逼近效果。

3 数值实现细节

3.1 算法要点

两种方法的核心区别在于：

- **傅里叶投影**：使用高密度采样（$M \gg N$）近似连续傅里叶积分，计算系数 $\hat{f}_k$，然后重构。
- **傅里叶插值**：仅使用 $N+1$ 个插值节点上的函数值，通过离散傅里叶变换直接得到系数 d_k，然后重构。

3.2 代码使用说明

- **软件环境**：MATLAB R2020a 及以上版本/北太天元。
- **运行方式**：将脚本 `hw4.m` 保存到当前目录，在命令窗口输入 `hw4` 运行。脚本会自动生成误差图和逼近曲线，并在命令窗口打印收敛阶表格。
- **切换测试函数**：修改脚本开头的 `f = u;` 为 `f = v;`，同时将 `fname` 改为 `'v'`。
- **修改模态数**：调整 `N_list` 的值（注意 N 必须为偶数）。

4 结果与讨论

4.1 函数 $v(x) = \sin(x/2)$

图 1 展示了投影与插值误差随 N 增大的变化。两种方法的 L^∞ 误差均随 N 增大而减小，但衰减速度较慢。理论分析表明，由于 $|\hat{v}_k| \sim 1/k^2$，截断后的最大模误差 $\sim \sum_{|k|>K} 1/|k| \sim 1/K$，$L^2$ 误差

$\sim \left(\sum_{|k|>K} 1/k^4\right)^{1/2} \sim K^{-3/2}$。表 2 中最大模误差的数值收敛阶约为 1.0，与理论一致（注意 $K = N/2$，故 $O(N^{-1})$）。同时注意到插值误差略大于投影误差，且在整个区间上分布较均匀。

图 3 给出了投影和插值得到的近似函数。当 $N = 4$ 时，近似曲线明显偏离真实函数；随着 N 增大，曲线逐渐逼近 $\sin(x/2)$，但在 $x = 2\pi$ 附近仍存在微小振荡。由于函数本身连续，该振荡并非经典的吉布斯跳跃现象，而是由端点导数不连续导致的局部慢收敛。

4.2 函数 $u(x) = \dfrac{1}{5 - 4\cos x}$

对于光滑函数 $u(x)$，傅里叶系数呈指数衰减，因此误差随 N 指数下降。图 2 表明，当 N 从 4 增加到 64 时，L^∞ 误差从约 10^{-1} 迅速降至 10^{-12} 以下。投影与插值的精度几乎相同，且在整个区间上误差分布非常均匀。

表 1 和表 2 列出了投影误差的 L^∞ 范数及其收敛阶。对于 u，每次 N 加倍，误差平方，收敛阶翻倍，这是指数收敛的典型特征；对于 v，收敛阶稳定在 1 附近，符合代数收敛 $O(N^{-1})$。

4.3 计算精度与成本对比

对于 $u(x)$，当 $N = 64$ 时，投影误差达到 10^{-10} 量级，插值误差略大（1.6×10^{-10}），两者基本一致。此时误差已远低于单精度极限，进一步增加 N 受限于函数值计算中的舍入误差。

从计算成本看，投影需要 $M = 2^{\lceil \log_2(10 \max(N,64)) \rceil}$ 个采样点，约为 N 的 10 倍；插值仅需 $N+1$ 个节点。若采用显式循环，投影的复杂度为 $O(KM)$，插值为 $O(K(N+1))$，投影约高出 $M/(N+1) \approx 10$ 倍。但若使用 FFT，投影的复杂度为 $O(M \log M)$，插值为 $O((N+1)\log(N+1))$，两者相当，且投影因采样点更多而略慢。实际应用中，对于光滑函数，插值已足够精确且更高效。

4.4 光滑性对逼近损失的影响

将光滑函数 $u(x)$ 与非光滑函数 $v(x)$ 的 L^∞ 误差并排对比（表 3），可以清晰看到光滑性的关键作用：

- **收敛速度差异**：N 从 4 增加到 64 时，u 的误差下降超过 9 个数量级，而 v 的误差仅下降约 1 个数量级。
- **相同 N 下的精度**：$N = 16$ 时 u 的误差已低于 10^{-2}，而 v 的误差仍在 10^{-2} 量级；$N = 32$ 时 u 的误差达到 10^{-5}，远超 v 在 $N = 64$ 时的精度。
- **实际应用启示**：对于光滑周期问题，傅里叶方法极为高效；对于仅连续或带尖点的函数，需要采用滤波、谱粘性法或非周期逼近方法来改善收敛性。

5 结论

本文通过数值实验比较了傅里叶投影和傅里叶插值对两个典型周期函数的逼近性能，得到以下结论：

1. 对于无限光滑的周期函数，傅里叶投影和插值均能实现指数收敛（误差随 N 指数下降），且两者精度相当。插值仅需 $N+1$ 个函数值，计算更经济。
2. 对于导数不连续的连续周期函数，傅里叶逼近仅具有代数收敛速度：L^∞ 误差 $\sim O(N^{-1})$，L^2 误差 $\sim O(N^{-3/2})$。此时端点附近出现局部慢收敛，但整体误差仍随 N 增大而减小。
3. 数值收敛阶与理论分析一致，验证了傅里叶级数截断误差对函数光滑性的敏感性。

本报告提供的 MATLAB 代码完整实现了两种方法，可作为进一步研究谱方法的基础。

A 图表附录

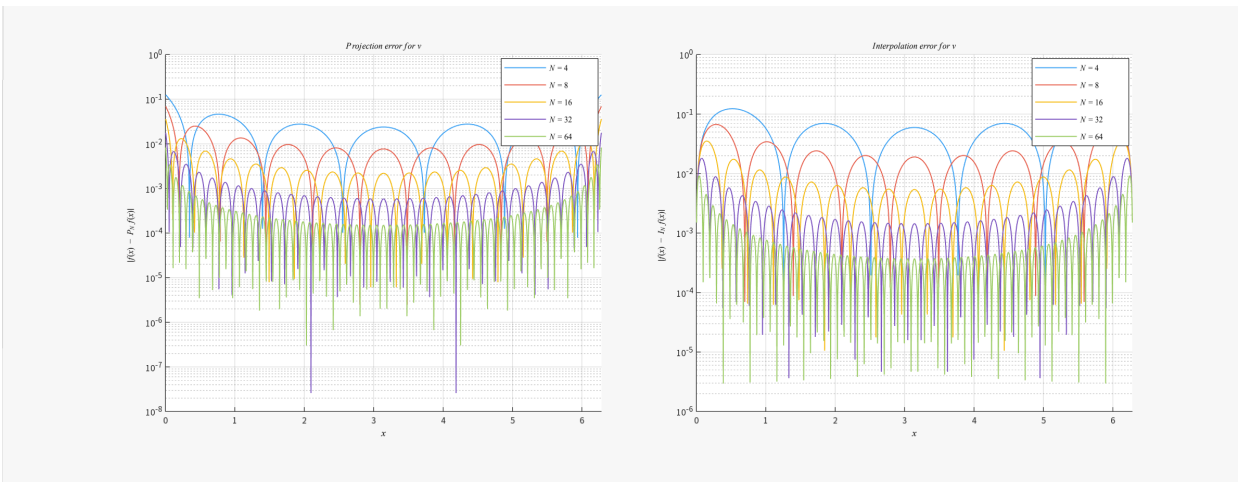

图 1: 函数 $v(x) = \sin(x/2)$ 的逼近误差（对数纵轴）。左：投影误差；右：插值误差。

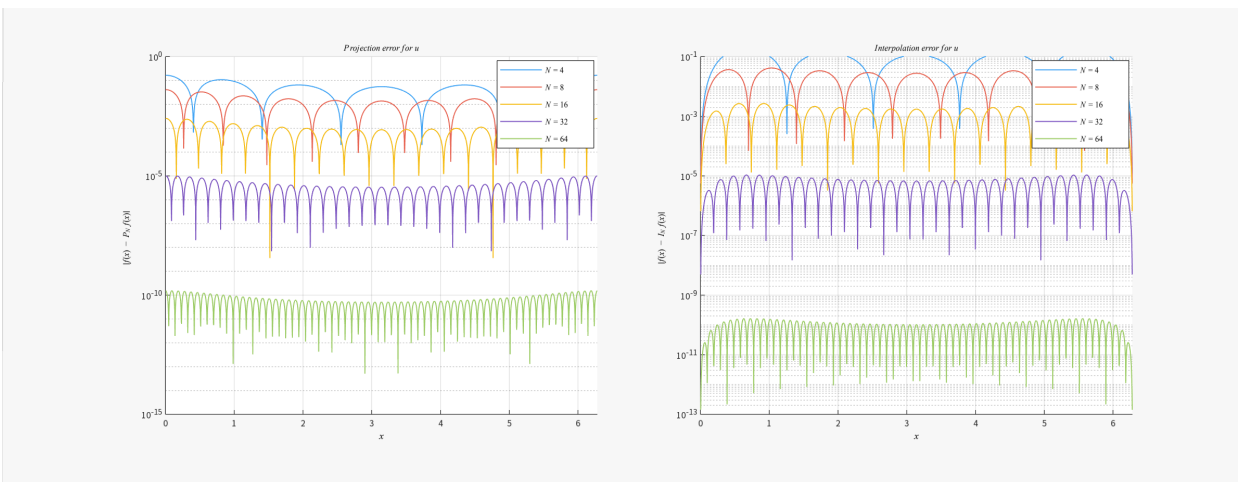

图 2: 函数 $u(x) = 1/(5 - 4\cos x)$ 的逼近误差（对数纵轴）。左：投影误差；右：插值误差。

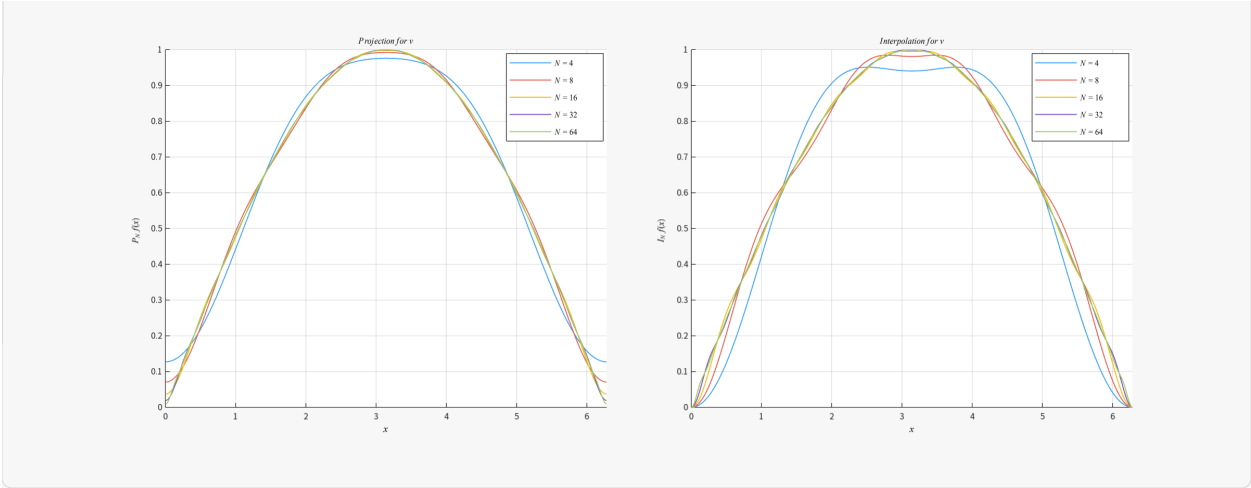

图 3: 函数 $v(x) = \sin(x/2)$ 的逼近曲线。左：投影；右：插值。

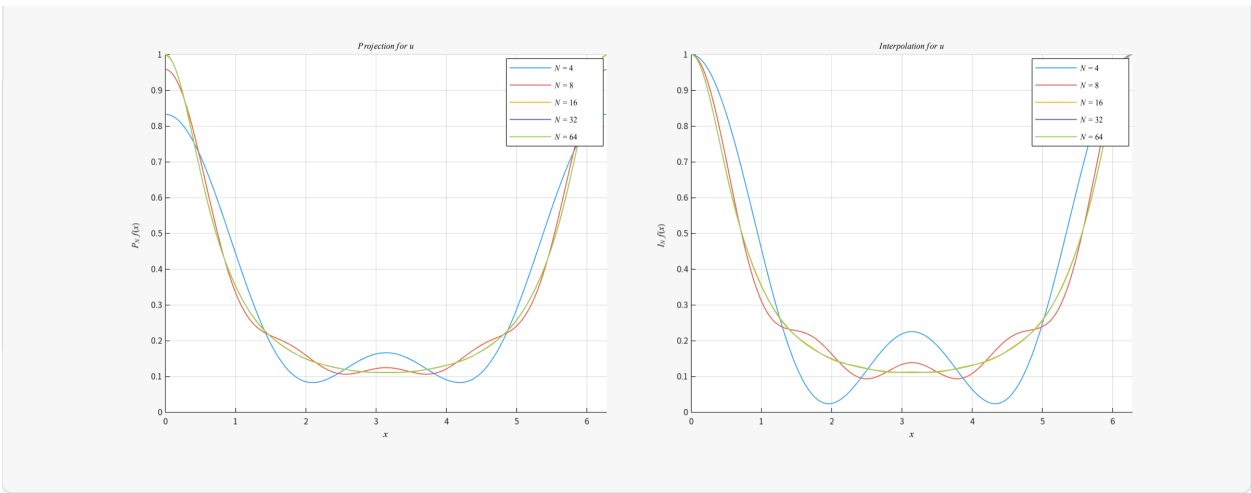

图 4: 函数 $u(x) = 1/(5 - 4\cos x)$ 的逼近曲线。左：投影；右：插值。

表 1: 函数 u 投影误差与插值误差的收敛性（L^∞ 误差）

N	Projection error	Order	Interpolation error	Order
4	1.6665×10^{-1}	—	1.8127×10^{-1}	—
8	4.1659×10^{-2}	2.0001	4.1832×10^{-2}	2.1155
16	2.6029×10^{-3}	4.0005	2.6906×10^{-3}	3.9586
32	1.0156×10^{-5}	8.0016	1.0757×10^{-5}	7.9665
64	1.5434×10^{-10}	16.006	1.6453×10^{-10}	15.997

表 2: 函数 v 投影误差与插值误差的收敛性（L^∞ 误差）

N	Projection error	Order	Interpolation error	Order
4	1.2575×10^{-1}	—	1.2318×10^{-1}	—
8	6.9166×10^{-2}	0.8625	6.6975×10^{-2}	0.8790
16	3.5881×10^{-2}	0.9468	3.5207×10^{-2}	0.9277
32	1.7729×10^{-2}	1.0171	1.8097×10^{-2}	0.9602
64	8.2412×10^{-3}	1.1052	9.1768×10^{-3}	0.9797

表 3: u 与 v 的投影 L^∞ 误差对比

N	u 误差	v 误差
4	1.67×10^{-1}	1.26×10^{-1}
8	4.17×10^{-2}	6.92×10^{-2}
16	2.60×10^{-3}	3.59×10^{-2}
32	1.02×10^{-5}	1.77×10^{-2}
64	1.54×10^{-10}	8.24×10^{-3}

B 计算机代码

以下为完整的 MATLAB 代码，包含主程序以及傅里叶投影、插值函数。

```
clear; close all;   % 清除工作区、命令窗口和图形
% clear; clc; close all;   % 清除工作区、命令窗口和图形

% 定义函数句柄
u = @(x) 1 ./ (5 - 4*cos(x));
v = @(x) sin(x/2);

% 密集网格（用于计算精确误差，画图用）
x_fine = linspace(0, 2*pi, 2000)';
x_fine = x_fine(2:end-1);   % 去掉 0,2 点，避免重复

N_list = [4, 8, 16, 32, 64];
colors = lines(length(N_list));

% 选择要处理的函数（可改为 u 或 v）
f = u;   % 或 v
fname = 'u';

figure('Position', [100 100 1200 500]);

% 子图1: 投影误差
```

```
subplot(1,2,1);
hold on;
for i = 1:length(N_list)
    N = N_list(i);
    P_f = FourierProjection(f, N, x_fine);
    err = abs(f(x_fine) - P_f);
    plot(x_fine, err, 'Color', colors(i,:), 'LineWidth', 1.5);
end
hold off;
xlim([0 2*pi]);
set(gca, 'YScale', 'log');
xlabel('x'); ylabel('|f(x) - P_N f(x)|');
title(sprintf('Projection error for %s', fname));
legend(arrayfun(@(n) sprintf('N=%d', n), N_list, 'UniformOutput', false));
grid on;

% 子图2: 插值误差
subplot(1,2,2);
hold on;
for i = 1:length(N_list)
    N = N_list(i);
    I_f = FourierInterpolation(f, N, x_fine);
    err = abs(f(x_fine) - I_f);
    plot(x_fine, err, 'Color', colors(i,:), 'LineWidth', 1.5);
end
hold off;
xlim([0 2*pi]);
set(gca, 'YScale', 'log');
xlabel('x'); ylabel('|f(x) - I_N f(x)|');
title(sprintf('Interpolation error for %s', fname));
legend(arrayfun(@(n) sprintf('N=%d', n), N_list, 'UniformOutput', false));
grid on;

figure('Position', [100 100 1200 500]);

% 子图1: 投影函数
subplot(1,2,1);
hold on;
for i = 1:length(N_list)
    N = N_list(i);
    P_f = FourierProjection(f, N, x_fine);
    err = abs(f(x_fine) - P_f);
    plot(x_fine, P_f, 'Color', colors(i,:), 'LineWidth', 1.5);
    % plot(x_fine, err, 'Color', colors(i,:), 'LineWidth', 1.5);
end
hold off;
```

```
xlim([0 2*pi]);
set(gca, 'YScale', 'linear');
xlabel('x'); ylabel('P_N f(x)');
title(sprintf('Projection for %s', fname));
legend(arrayfun(@(n) sprintf('N=%d', n), N_list, 'UniformOutput', false));
grid on;

% 子图2: 插值函数
subplot(1,2,2);
hold on;
for i = 1:length(N_list)
    N = N_list(i);
    I_f = FourierInterpolation(f, N, x_fine);
    err = abs(f(x_fine) - I_f);
    plot(x_fine, I_f, 'Color', colors(i,:), 'LineWidth', 1.5);
end
hold off;
xlim([0 2*pi]);
set(gca, 'YScale', 'linear');
xlabel('x'); ylabel('I_N f(x)');
title(sprintf('Interpolation for %s', fname));
legend(arrayfun(@(n) sprintf('N=%d', n), N_list, 'UniformOutput', false));
grid on;

function P_f = FourierProjection(f, N, x_fine)
    % 手搓版: 不使用 fft, 直接计算傅里叶系数
    K = N/2;                    % 最大频率
    M = 2^nextpow2(10*max(N,64));  % 采样点数（足够密）
    x_sample = linspace(0, 2*pi, M+1)';
    x_sample = x_sample(1:end-1);
    f_sample = f(x_sample);
    dx = 2*pi / M;

    % 计算傅里叶系数 hat{f}_k (k = -K .. K)
    coeffs = zeros(1, 2*K+1);
    for idx = 1:2*K+1
        k = idx - K - 1;        % k 从 -K 到 K
        % DFT 求和（矩形法则, 对于光滑周期函数精度极高）
        sum_val = 0;
        for j = 1:M
            sum_val = sum_val + f_sample(j) * exp(-1i * k * x_sample(j));
        end
        coeffs(idx) = sum_val / M;   % 除以 M 得到正确的系数
    end

    % 在 x_fine 上重构
```

```
    P_f = zeros(size(x_fine));
    for idx = 1:2*K+1
        k = idx - K - 1;
        P_f = P_f + coeffs(idx) * exp(1i * k * x_fine);
    end
    P_f = real(P_f);
end

function I_f = FourierInterpolation(f, N, x_fine)
    L = N + 1;
    x_nodes = (0:N) * 2*pi / L;
    f_nodes = f(x_nodes);
    K = N/2;

    % 计算插值系数 d_k (k = -K .. K)
    d = zeros(1, 2*K+1);
    for idx = 1:2*K+1
        k = idx - K - 1;
        sum_val = 0;
        for j = 1:L
            sum_val = sum_val + f_nodes(j) * exp(-1i * k * x_nodes(j));
        end
        d(idx) = sum_val / L;
    end

    % 重构
    I_f = zeros(size(x_fine));
    for idx = 1:2*K+1
        k = idx - K - 1;
        I_f = I_f + d(idx) * exp(1i * k * x_fine);
    end
    I_f = real(I_f);
end
```